

Министерство науки и высшего образования Российской Федерации
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Институт прикладной математики и компьютерных наук

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 2

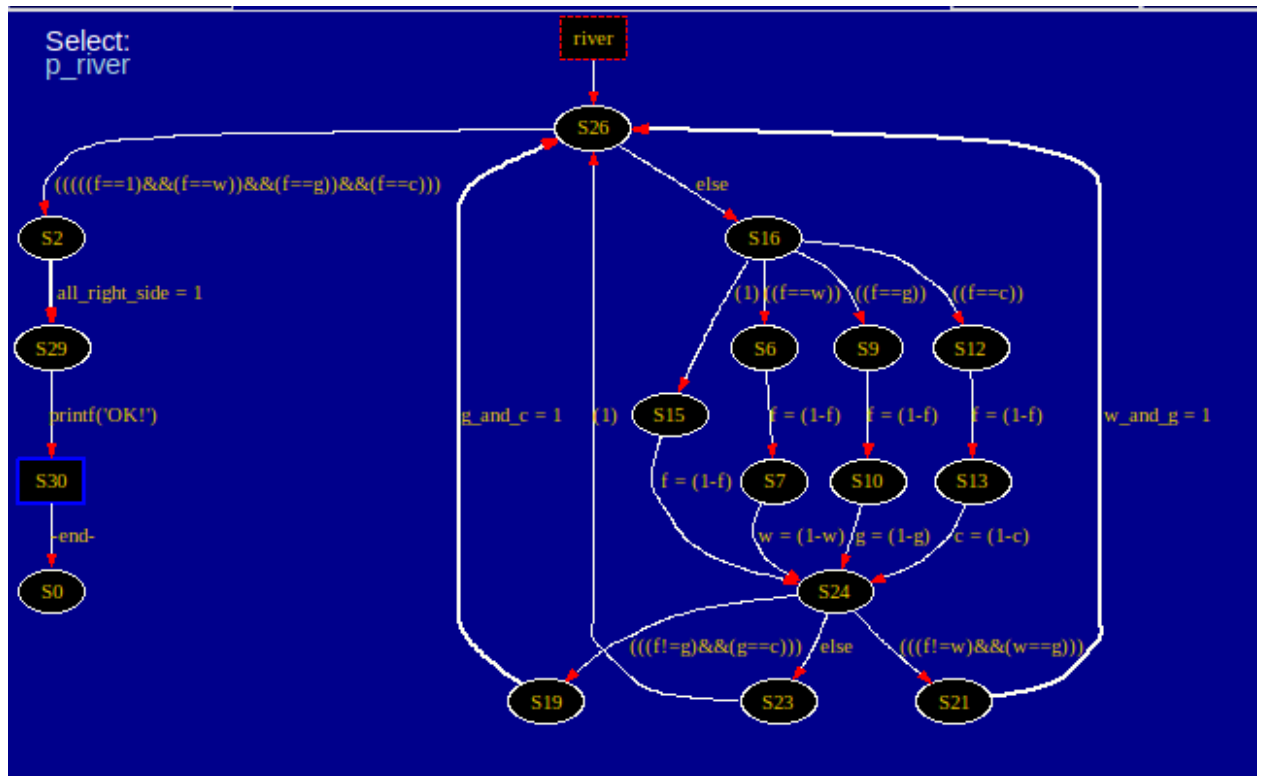
по дисциплине «Методы верификации»

Выполнил:
студент группы № 932223
Захаров А.А.

Проверил:
Преподаватель
Лапутенко А. В.

Томск – 2026

Решить задачу о волке, козе и капусте одним из способов. Проанализировать получившуюся траекторию (код есть в книге: И.В. Шошмина, Ю.Г. Карпов Введение в язык Promela и систему комплексной верификации SPIN)



1. Изначально капуста, волк, фермер и коза на одном и том же берегу (S26); шаги 2-4 помещены в блок `do ... od`;
2. Выходим из цикла `do` по условию `(f==1) && (f == w) && (f == g) && (f==c) -> all_right_side = true`;
3. Иначе: если несколько из условий `(f==w)`; `(f==g)`; или `(f==c)` выполнимы, то недетерминированно выполняется одна из последовательностей действий, соответствующая выполненному условию:
 - a. `f = (1-f); w = (1-w);` // Фермер забирает волка на другой берег
 - b. `f = (1-f); g = (1-g);` // Фермер забирает козу на другой берег
 - c. `f = (1-f); c = (1-c);` // Фермер забирает капусту на другой берег
 - d. `f = (1-f);` // Фермер просто уплывает на другой берег
4. Аналогично шагу 3, по условиям `(f!=g && g==c)`; или `(f!=w && w==g)` выполняется
 - a. `g_and_c = true;` // Коза и капуста остались на одном берегу
 - b. `w_and_g = true;` // Волк и коза остались на одном берегу
 - c. `else skip;` // иначе ничего не делаем
5. Переход на шаг 2.

```

#define fin (all_right_side ==true);
#define gc (g_and_c == false);
#define wg (w_and_g == false) ;
bool all_right_side, w_and_g, g_and_c;

active proctype river()
{
    bit f = 0, w=0, g = 0, c = 0;
    do
        :: (f==1) && (f == w) && (f == g) && (f==c) -> all_right_side = true; break;
        :: else ->
            if
                :: (f == w) -> f = 1-f; w = 1-w;
                :: (f == g) -> f = 1-f; g = 1-g;
                :: (f == c) -> f = 1-f; c = 1- c;
                :: (true) -> f = 1-f;
            fi;
            if
                ::(f != g) && (g == c) -> g_and_c = true;
                :: (f != w) && (w == g) -> w_and_g=true;
                :: else -> skip
            fi;
    od;
    printf("OK!");
}

```

```

1:      proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
2:      proc 0 (river:1) koza_volk_kapusta.pml:16 (state 14) [(1)]
3:      proc 0 (river:1) koza_volk_kapusta.pml:16 (state 15) [f = (1-f)]
5:      proc 0 (river:1) koza_volk_kapusta.pml:20 (state 20) [(((f!=w)&&(w==g)))]
6:      proc 0 (river:1) koza_volk_kapusta.pml:20 (state 21) [w_and_g = 1]
9:      proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
10:     proc 0 (river:1) koza_volk_kapusta.pml:16 (state 14) [(1)]
11:     proc 0 (river:1) koza_volk_kapusta.pml:16 (state 15) [f = (1-f)]
13:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 22) [else]
14:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 23) [(1)]
17:     proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
18:     proc 0 (river:1) koza_volk_kapusta.pml:14 (state 8) [((f==g))]
19:     proc 0 (river:1) koza_volk_kapusta.pml:14 (state 9) [f = (1-f)]
20:     proc 0 (river:1) koza_volk_kapusta.pml:14 (state 10) [g = (1-g)]
22:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 22) [else]
23:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 23) [(1)]
26:     proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
27:     proc 0 (river:1) koza_volk_kapusta.pml:16 (state 14) [(1)]
28:     proc 0 (river:1) koza_volk_kapusta.pml:16 (state 15) [f = (1-f)]
30:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 22) [else]
31:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 23) [(1)]
34:     proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
35:     proc 0 (river:1) koza_volk_kapusta.pml:15 (state 11) [((f==c))]
36:     proc 0 (river:1) koza_volk_kapusta.pml:15 (state 12) [f = (1-f)]
37:     proc 0 (river:1) koza_volk_kapusta.pml:15 (state 13) [c = (1-c)]
39:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 22) [else]
40:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 23) [(1)]
43:     proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
44:     proc 0 (river:1) koza_volk_kapusta.pml:16 (state 14) [(1)]
45:     proc 0 (river:1) koza_volk_kapusta.pml:16 (state 15) [f = (1-f)]
47:     proc 0 (river:1) koza_volk_kapusta.pml:19 (state 18) [(((f!=g)&&(g==c)))]
48:     proc 0 (river:1) koza_volk_kapusta.pml:19 (state 19) [g_and_c = 1]
51:     proc 0 (river:1) koza_volk_kapusta.pml:11 (state 4) [else]
52:     proc 0 (river:1) koza_volk_kapusta.pml:13 (state 5) [((f==w))]
53:     proc 0 (river:1) koza_volk_kapusta.pml:13 (state 6) [f = (1-f)]
54:     proc 0 (river:1) koza_volk_kapusta.pml:13 (state 7) [w = (1-w)]
56:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 22) [else]
57:     proc 0 (river:1) koza_volk_kapusta.pml:21 (state 23) [(1)]
60:     proc 0 (river:1) koza_volk_kapusta.pml:10 (state 1) [((((f==1)&&(f==w))&&(f==g))&&(f==c)))]
61:     proc 0 (river:1) koza_volk_kapusta.pml:10 (state 2) [all_right_side = 1]
OK! 63: proc 0 (river:1) koza_volk_kapusta.pml:24 (state 29) [printf('OK!')]
63:     proc 0 (river:1) terminates
1 processes created

```

[variable values, step 61]

```

all_right_side = 1
g_and_c = 1
river(0):c = 1
river(0):f = 1
river(0):g = 1
river(0):w = 1
w_and_g = 1

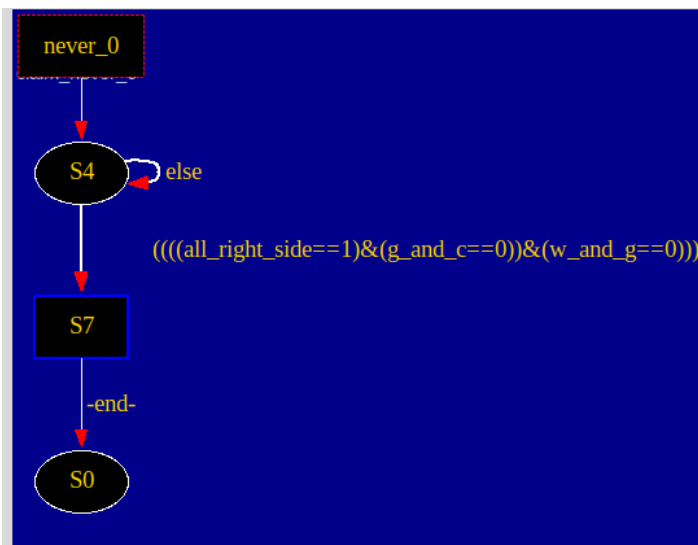
```

Далее во вкладке “Verification” запускаем Run

```
never{  
  do  
    :: (fin & gc & wg) -> break;  
    :: else  
  od;  
}
```

Ошибочным состоянием здесь является состояние, в которое можем попасть только по условию `fin && gc && wg == true`, то есть по условию

```
((all_right_side == true) &&  
(g_and_c == false) &&  
(w_and_g == false)) == true
```



State-vector 12 byte, depth reached 80, errors: 0

238 states, stored

77 states, matched

315 transitions (= stored+matched)

0 atomic steps

hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):

0.009 equivalent memory usage for states (stored*(State-vector + overhead))

0.289 actual memory usage for states

128.000 memory used for hash table (-w24)

0.534 memory used for DFS stack (-m10000)

128.730 total actual memory usage

unreached in proctype river

(0 of 30 states)

unreached in claim never_0

koza_volk_kapusta.pml:31, state 4, "((((all_right_side==1)&(g_and_c==0))&(w_and_g==0)))"

koza_volk_kapusta.pml:31, state 4, "else"

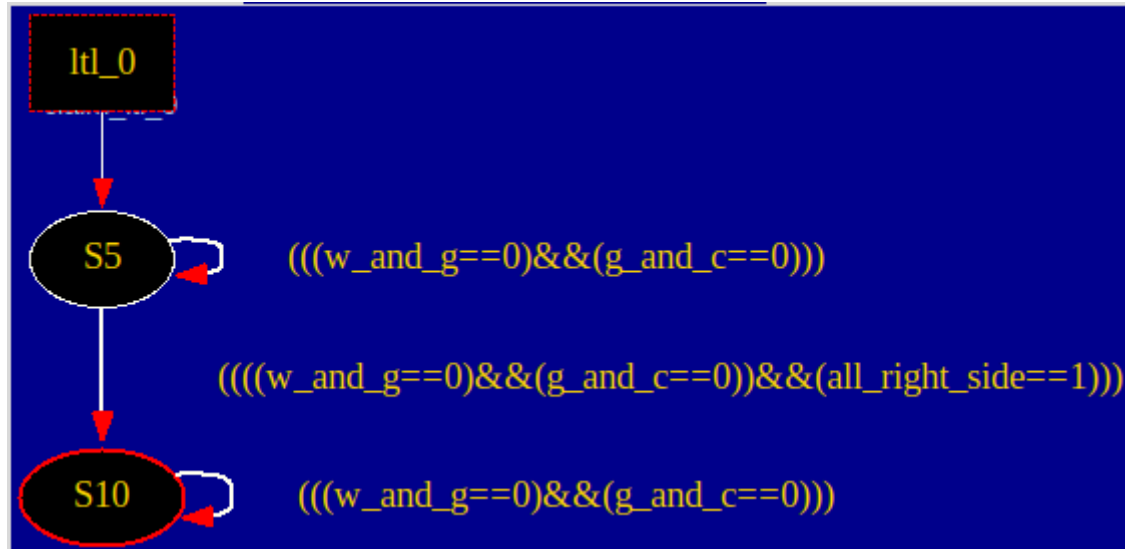
koza_volk_kapusta.pml:34, state 7, "-end-"

(2 of 7 states)

pan: elapsed time 0 seconds

No errors found -- did you verify all claims?

```
ltl {!( <> fin && [] ( wg && gc ) ) }
```



```
not( eventually ( all_right_side ==true ) &&
always ( (g_and_c == false) && (w_and_g == false)))
```

То есть **отрицание**(случится `(all_right_side ==true)`) И **всегда_выполняется**(`((g_and_c == false) && (w_and_g == false))`)

Если выполнится отрицание этого условия – то обнаружена ошибка.

```

State-vector 20 byte, depth reached 80, errors: 0
  238 states, stored
   77 states, matched
  315 transitions (= stored+matched)
    0 atomic steps
hash conflicts:      0 (resolved)

Stats on memory usage (in Megabytes):
  0.011  equivalent memory usage for states (stored*(State-vector + overhead))
  0.288  actual memory usage for states
128.000 memory used for hash table (-w24)
  0.534  memory used for DFS stack (-m10000)
128.730 total actual memory usage

unreached in proctype river
  (0 of 30 states)
unreached in claim ltl_0
  _spin_nvr.tmp:3, state 5, "(((w_and_g==0)&&(g_and_c==0))&&(all_right_side==1)))"
  _spin_nvr.tmp:3, state 5, "(((w_and_g==0)&&(g_and_c==0)))"
  _spin_nvr.tmp:8, state 10, "(((w_and_g==0)&&(g_and_c==0)))"
  _spin_nvr.tmp:10, state 13, "-end-"
  (3 of 13 states)

pan: elapsed time 0 seconds
No errors found -- did you verify all claims?
  
```